

KT 사례기반 AI 교육

- VOC 고객 해지 예측 모델링

1. AIDU 기본 사용법 및 데이터 분석 준비하기(완료)
2. 데이터 탐색하기(완료)
3. 데이터 전처리하기(완료)
4. 머신러닝 모델 구현(완료)

5. 딥러닝 심층신경망 모델 구현

```
In [ ]: # 코드실행시 경고 메시지 무시
import warnings
warnings.filterwarnings(action='ignore')
```

```
In [ ]: # 필요 라이브러리 임포트 하기
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [ ]: #!pip install seaborn
```

Chapter 5. 딥러닝 심층신경망 모델 구현

이번시간에는 Python을 활용한 AI 모델링에서 딥러닝에 대해 실습해 보겠습니다.
여기서는 딥러닝 모델 DNN에 대해 코딩하여 모델 구축해 보겠습니다.
이론보다 실습이 더 많은 시간과 노력이 투자 되어야 합니다.

1. 데이터 가져오기
2. 데이터 전처리 (완료)
3. Train, Test 데이터셋 분할
4. 데이터 정규화 (MinMaxScaling, StandardScaling)
5. 딥러닝 모델 구현
6. 딥러닝 성능 평가

1. 데이터 가져오기

```
In [ ]: # voc_rcp_practice3.csv 데이터 읽어 들이기
df = pd.read_csv("../voc_rcp_practice3.csv")
```

```
In [ ]: df.head()
```

2. 데이터 전처리하기 - 완료

3. Train, Test 데이터셋 분할

- ① Feature / Target 데이터 분리
- ② Train / Test 데이터 Set 나누기

① Feature / Target 데이터 분리

- Feature는 X, Target은 y로 분리

```
In [ ]: # Feature는 Target('trm_yn') 제외한 나머지
X = df.drop(columns=['trm_yn']).values

# Target은 해지여부('trm_yn')
y = df['trm_yn'].values
```

```
In [ ]: print(X.shape, y.shape)
```

② Train / Test 데이터 Set 나누기

- X, y 값을 가지고 7:3 비율로 Train, Test 을 나누세요.
- train_test_split 시 stratify 옵션 사용

```
In [ ]: # 라이브러리 임포트
from sklearn.model_selection import train_test_split
```

```
In [ ]: # 데이터 Set 나누기 (Train:Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y,
```

```
In [ ]: print(X_train.shape, y_train.shape)
```

4. 데이터 스케일링(MinMaxScaler)

```
In [ ]: # 라이브러리 임포트
from sklearn.preprocessing import MinMaxScaler

# 스케일러 호출
scaler = MinMaxScaler()

# 스케일링 (Train, Test)
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

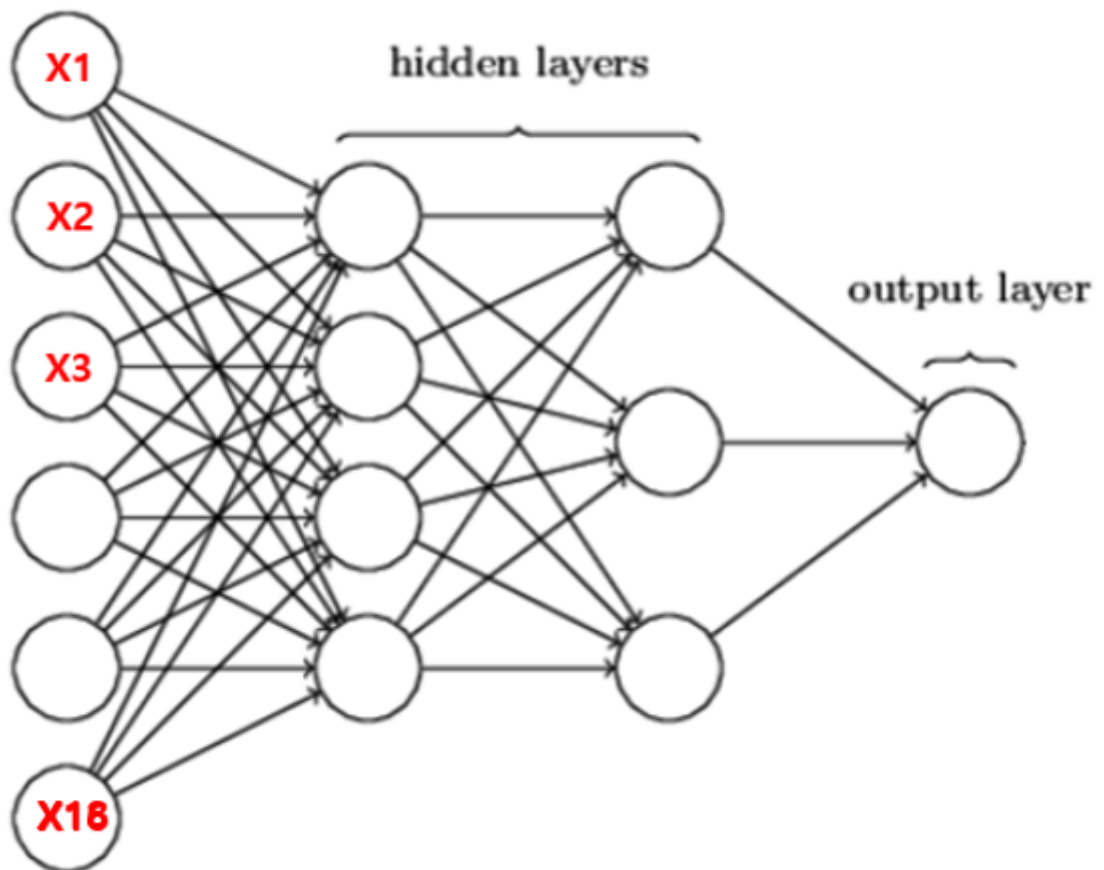
5. 딥러닝 모델 구현 (DNN)

- ① 모델 생성하기 : Sequential 모델
- ② 모델 컴파일 하기
- ③ 모델 학습하기
- ④ Callbacks 함수 설정하기 : EarlyStopping, ModelCheckpoint

① 모델 생성하기 : Sequential 모델

- 하이퍼 파라미터 설정 : batch_size = 1024, epochs = 20
- hidden layer 3개 이상으로 모델을 구성
- 과적합 방지하는 dropout을 설정하세요.

A. 이진 분류용 DNN layer



[출처] <https://subscription.packtpub.com/book/data/9781788995207/1/ch01lv1sec03/deep-learning-intuition>

• 라이브러리 임포트

```
In [ ]: # 라이브러리 임포트
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Dropout
```

• 모델 선언

```
In [ ]: # 80개 input layer
# unit 4개 hidden layer
# unit 3개 hidden layer
```

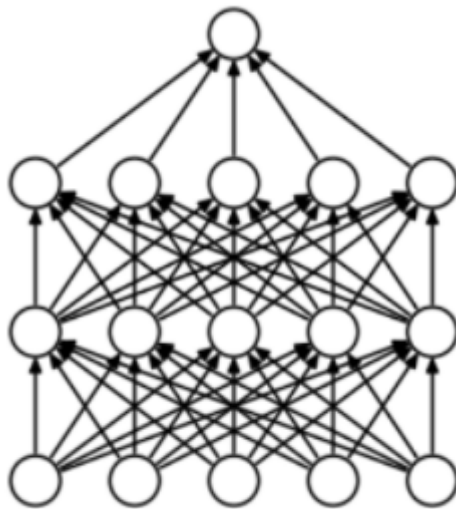
```
# 1개 output layer : 이진분류

model = Sequential()
model.add(Dense(4, activation='relu', input_shape=(80,)))
model.add(Dense(3, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

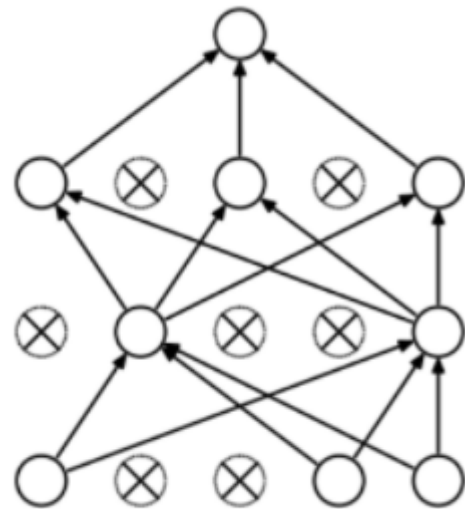
- 모델 확인

```
In [ ]: model.summary()
```

[Dropout : 과적합 방지]



(a) Standard Neural Net



(b) After applying dropout.

[출처] <https://medium.com/@amarbudhiraja/https-medium-com-amarbudhiraja-learning-less-to-learn-better-dropout-in-deep-machine-learning-74334da4bfc5>

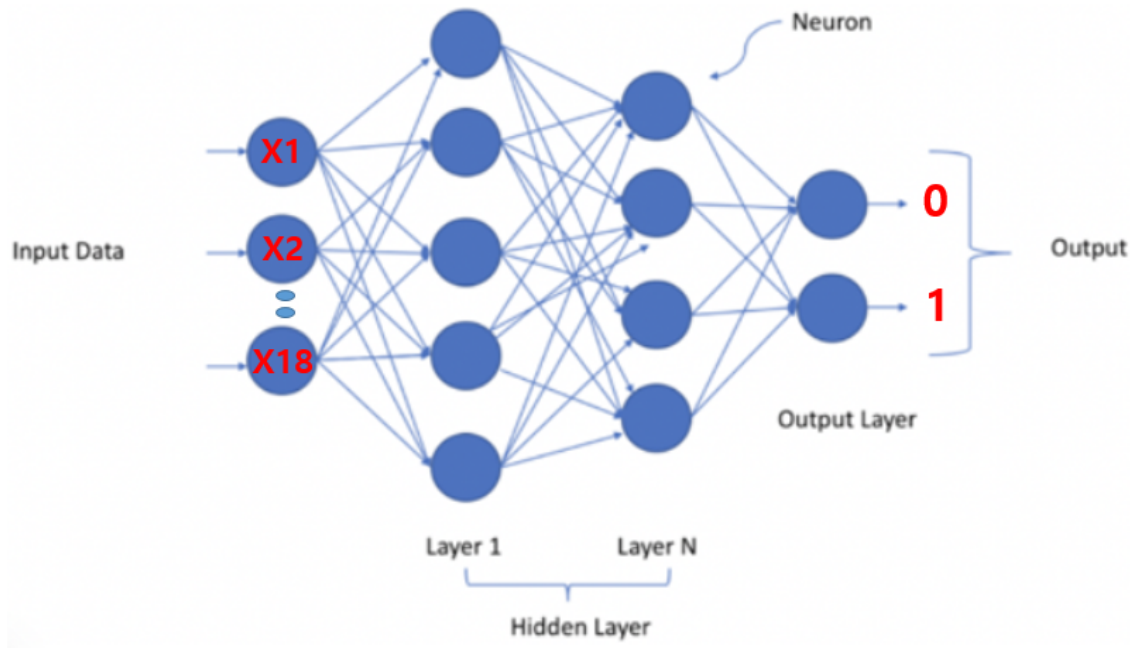
```
In [ ]: # 80개 input layer
# unit 4개 hidden layer
# dropout
# unit 3개 hidden layer
# dropout
# 1개 output layer : 이진분류

model2 = Sequential()
model2.add(Dense(32, activation='relu', input_shape=(80,)))
model2.add(Dropout(0.3))
model2.add(Dense(16, activation='relu'))
model2.add(Dropout(0.3))
model2.add(Dense(1, activation='sigmoid'))
```

- 모델 확인

```
In [ ]: model2.summary()
```

B. 다중 분류용 DNN layer



[출처] <https://www.educba.com/dnn-neural-network/>

• 모델 선언

```
In [ ]: # 18개 input layer
# unit 5개 hidden layer
# dropout
# unit 4개 hidden layer
# dropout
# 2개 output layer : 다중분류

model3 = Sequential()
model3.add(Dense(5, activation='relu', input_shape=(80,)))
model3.add(Dropout(0.3))
model3.add(Dense(4, activation='relu'))
model3.add(Dropout(0.3))
model3.add(Dense(2, activation='softmax'))
```

• 모델 확인

```
In [ ]: model3.summary()
```

② 모델 컴파일 하기

A. 이진 분류 모델

```
In [ ]: model.compile(optimizer='adam',
                      loss='binary_crossentropy',
                      metrics=['accuracy'])
```

B. 다중 분류 모델

```
In [ ]: model3.compile(optimizer='adam',
                      loss='sparse_categorical_crossentropy',
```

```
metrics=['accuracy'])
```

```
# 원-핫 인코딩이 안되어 있을 경우 다중 분류 모델의 loss는 'sparse_categorical_crossentropy'
```

③ 모델 학습하기

```
In [ ]: history = model.fit(X_train, y_train,
                        validation_data=(X_test, y_test),
                        epochs=20,
                        batch_size=16)
```

④ Callbacks 함수 설정하기 : EarlyStopping, ModelCheckpoint

- 라이브러리 импорт

```
In [ ]: # 라이브러리 импорт
        from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

- Callback 선언

A. EarlyStopping

모델을 더 이상 학습을 못할 경우(loss, metric 등의 개선이 없을 경우), 학습 도중 미리 학습을 종료시키는 콜백 함수입니다.

monitor : EarlyStopping의 기준이 되는 값('val_loss'를 입력하면 val_loss가 더 이상 감소되지 않을 경우 적용)
 min_delta : 개선된 것으로 간주하기 위한 최소한의 변화량
 patience : monitor되는 값의 개선이 없을 때 최적의 값을 기준으로 몇 번의 epoch를 더 진행할 지 정하는 값
 verbose : 화면에 상태 표시
 mode : monitor가 최소가 되어야 하는 지, 최대가 되어야 하는지 ('val_accuracy'는 max, 'val_loss'는 min)

```
In [ ]: es = EarlyStopping(monitor='val_loss', patience=3, mode='min', verbose=1)
```

B. ModelCheckpoint

모델을 저장할 때 사용되는 콜백 함수입니다.

filepath : 모델을 저장할 경로를 입력합니다. '파일명.h5'
 모델을 저장할 때, 기준이 되는 값을 지정합니다. ('val_loss'를 입력하면 validation set의 loss가 가장 적을 때 저장)
 verbose : 화면에 상태 표시
 save_best_only : True, False
 mode : auto로 할 경우, 모델이 알아서 min, max 판단하여 모델 저장

```
In [ ]: mc = ModelCheckpoint('my_checkpoint.ckpt', monitor='val_loss', save_best_only=True, v
```

- **Callback을 설정하고 학습하기**

```
In [ ]: model2.compile(optimizer='adam',
                    loss='binary_crossentropy',
                    metrics=['accuracy'])
```

```
In [ ]: history = model2.fit(X_train, y_train,
                            validation_data=(X_test, y_test),
                            epochs=20,
                            callbacks=[es, mc],
                            batch_size=16,
                            verbose=1)
```

```
In [ ]: model2.load_weights("/content/my_checkpoint.ckpt")
```

```
In [ ]: model2.save("my_model.h5")
```

⑤ 모델 성능 평가(시각화)

- 성능 시각화

```
In [ ]: performance = pd.DataFrame(model2.history.history)
```

```
In [ ]: performance.head()
```

```
In [ ]: plt.plot(performance[['loss', 'val_loss']])
plt.legend(['loss', 'val_loss'])
plt.xlabel('Epochs')
plt.show()
```

```
In [ ]: plt.plot(performance[['loss', 'val_loss', 'accuracy', 'val_accuracy']])
plt.legend(['loss', 'val_loss', 'accuracy', 'val_accuracy'])
plt.xlabel('Epochs')
plt.show()
```

[실습] 딥러닝 모델 구현하기

[Q] 아래의 조건으로 DNN 모델을 구현해 보세요.

- **히든 레이어를 3개**로 모델을 구성하고 과적합 방지를 위한 **dropout** 설정
- 모델의 optimizer는 'adam'을 사용하고 metrics는 'accuracy'로 하여 컴파일하세요.
- **epochos**는 20으로 **batch_size**는 32로 설정
- **EarlyStopping** 콜백으로 정해진 epoch 동안 모니터링 지표가 향상되지 않을 때 훈련 중단(
monitor는 'val_accuracy', patience는 5 설정)
- **ModelCheckpoing** 콜백으로 validation performance가 가장 좋은 모델을 '**이름.h5**' 파일로 저장하세요.
- 모델 학습 후 결과를 **history_q** 변수에 저장하세요.

```
In [ ]:
```

```

# 여기에 입력하세요
# 라이브러리 임포트
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# input layer 확인
X_train.shape

# 모델 선언 (반수 : model_q)
model_q = Sequential()
model_q.add(Dense(32, activation='relu', input_shape=(80,)))
model_q.add(Dropout(0.3))
model_q.add(Dense(16, activation='relu'))
model_q.add(Dropout(0.3))
model_q.add(Dense(8, activation='relu'))
model_q.add(Dropout(0.3))
model_q.add(Dense(1, activation='sigmoid'))

# 모델 확인
model_q.summary()

# 모델 컴파일하기
model_q.compile(optimizer='adam',
                loss='binary_crossentropy',
                metrics=['accuracy'])

# callbacks 함수 설정
es = EarlyStopping(monitor='val_accuracy', patience=5, mode='max', verbose=1)
mc = ModelCheckpoint('my_checkpoint_q.ckpt', monitor='val_accuracy', save_best_only=True)

# 모델 훈련
history = model_q.fit(X_train, y_train,
                    validation_data=(X_test, y_test),
                    epochs=20,
                    callbacks=[es, mc],
                    batch_size=32,
                    verbose=1)

model_q.load_weights("my_checkpoint_q.ckpt")
model_q.save("석재민.h5")

```

[Q] 구현한 DNN 모델의 학습 결과를 'acc' 와 'val-acc'의 관계 그래프를 아래의 조건으로 표시하세요.

- 그래프 제목은 'Accuracy'로 표시하세요.
- x축은 'Epochs' y축은 'Acc'로 표시하세요.
- 범례를 'acc, 'val_acc'로 표시하세요.

In []:

```

# 여기에 입력하세요.
performance_q = pd.DataFrame(model_q.history.history)

plt.plot(performance_q[['accuracy', 'val_accuracy']])
plt.legend(['accuracy', 'val_accuracy'])
plt.xlabel('Epochs')
plt.ylabel('Acc')
plt.show()

```

In []: